

Predictive Analytics Assignment - 2

Shreyanshi Adak - 745

2026-02-05

1. Problem to demonstrate that the population regression line is fixed, but least square regression line varies

Suppose the population regression line is given by $Y = 2 + 3x$, while the data comes from the model $y = 2 + 3x + \epsilon$.

Suppose the population regression line is given by:

$$Y = 2 + 3x$$

while the data comes from the model:

$$y = 2 + 3x + \epsilon$$

where:

$$\epsilon \sim N(0, 4^2)$$

We take $n = 50$ and seed = 123.

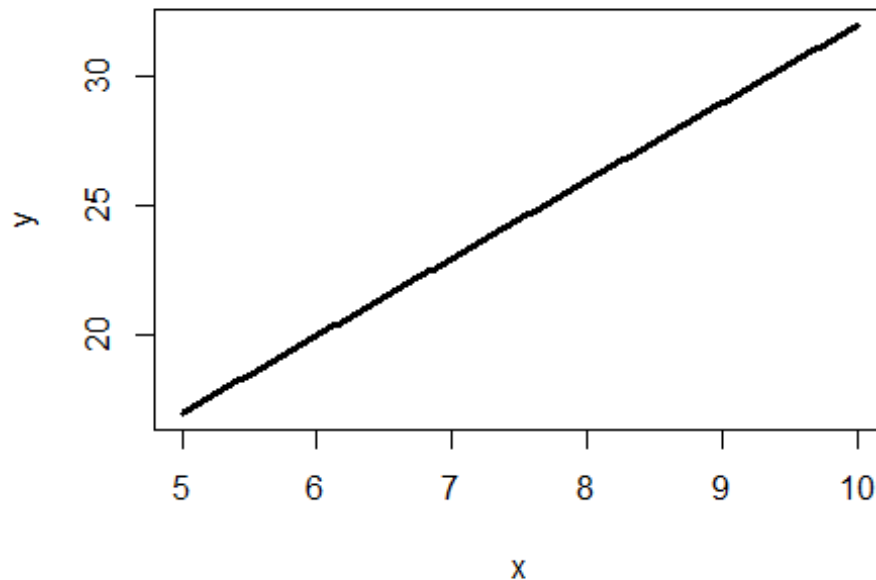
Step 1: Graph the Population Regression Line

For $x \in [5, 10]$, the population regression line is plotted below.

```
set.seed(123)
# Population regression line
x_vals <- seq(5, 10, length.out = 200)
y_pop <- 2 + 3 * x_vals

plot(x_vals, y_pop, type="l", lwd=3,
     main="Population Regression Line and Sample Regression Lines",
     xlab="x", ylab="y")
```

Population Regression Line and Sample Regression Line



Step 2–3: Generate Sample Data and Report Least Squares Regression Line

We generate:

- $x_i \sim \text{Uniform}(5, 10)$
- $\epsilon_i \sim N(0, 4^2)$
- $y_i = 2 + 3x_i + \epsilon_i$

Then we estimate the regression line.

```
n <- 50

x <- runif(n, 5, 10)
eps <- rnorm(n, mean=0, sd=4)
y <- 2 + 3*x + eps

model <- lm(y ~ x)
summary(model)

##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -9.0231 -2.2314 -0.2627  2.1970  8.7445
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.09639    2.82610  -0.034    0.973
## x           3.30540    0.36519   9.051 5.96e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.761 on 48 degrees of freedom
## Multiple R-squared:  0.6306, Adjusted R-squared:  0.6229
## F-statistic: 81.93 on 1 and 48 DF, p-value: 5.962e-12
```

The estimated least squares regression line is:

$$\hat{y} = -0.09639 + 3.30x$$

Step 4: Repeat the Simulation 5 Times

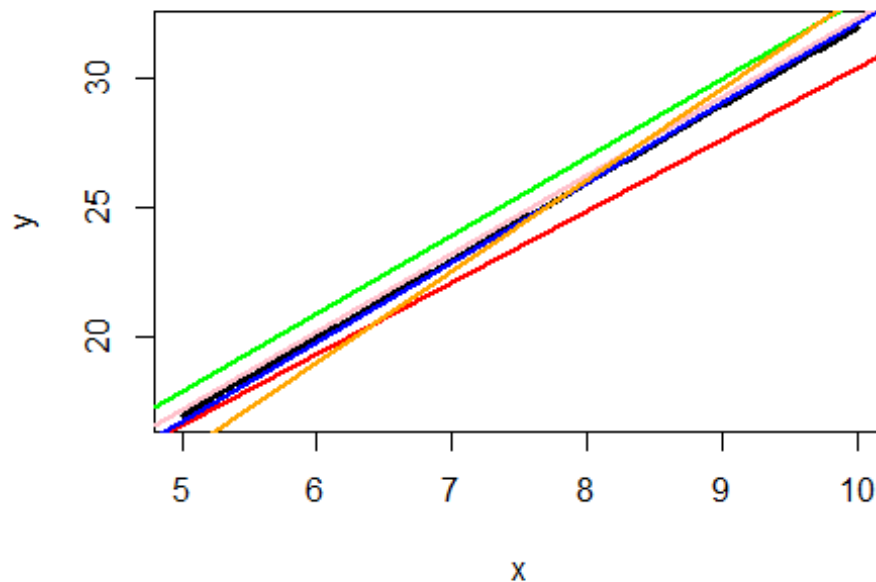
We repeat Steps 2–3 five times and plot the regression lines.

```
plot(x_vals, y_pop, type="l", lwd=3,
     main="Population Line vs 5 Sample Regression Lines",
     xlab="x", ylab="y")

# Repeat simulation 5 times
for(i in 1:5){
  x <- runif(n, 5, 10)
  eps <- rnorm(n, 0, 4)
  y <- 2 + 3*x + eps

  model <- lm(y ~ x)
  abline(model, lwd=2, col=c("red", "blue", "green", "pink", "orange")[i])
}
```

Population Line vs 5 Sample Regression Lines



Interpretation

The population regression line is fixed at:

$$Y = 2 + 3x$$

However, each time we generate a new random sample, the least squares regression line changes slightly due to random error.

Thus, the estimated regression line varies across samples even though the true population line remains constant.

2 Problem to demonstrate that $\hat{\beta}_0$ and $\hat{\beta}$ minimises RSS

Step 1: Generate Data

We generate:

- $x_i \sim \text{Uniform}(5,10)$
- Mean-center the values
- $\epsilon_i \sim N(0,1)$

$$y_i = 2 + 3x_i + \epsilon_i$$

```
n <- 50
```

```
x <- runif(n, 5, 10)

# Mean centering
x <- x - mean(x)

eps <- rnorm(n, 0, 1)

y <- 2 + 3*x + eps
```

Step 2: Obtain Least Squares Estimates

Assume:

$$y_i = \beta_0 + \beta x_i + \epsilon_i$$

```
model2 <- lm(y ~ x)
summary(model2)

##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.05069 -0.51840 -0.00825  0.54497  1.66532
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.05141    0.11869   17.28  <2e-16 ***
## x            2.94120    0.08165   36.02  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8393 on 48 degrees of freedom
## Multiple R-squared:  0.9643, Adjusted R-squared:  0.9636
## F-statistic: 1298 on 1 and 48 DF, p-value: < 2.2e-16

beta0_hat <- coef(model2)[1]
beta1_hat <- coef(model2)[2]

beta0_hat

## (Intercept)
##      2.051413

beta1_hat

##           x
## 2.941197
```

Thus the least squares estimates are:

$$\widehat{\beta}_0 = 2.051413, \hat{\beta} = 2.941197$$

Step 3: Grid Search to Find Minimum RSS

RSS is:

$$RSS = \sum_{i=1}^n (y_i - \beta_0 - \beta x_i)^2$$

We compute RSS over many combinations of (β_0, β) .

```
beta0_grid <- seq(beta0_hat - 2, beta0_hat + 2, length=50)
beta1_grid <- seq(beta1_hat - 2, beta1_hat + 2, length=50)

rss_values <- matrix(NA, nrow=50, ncol=50)

for(i in 1:50){
  for(j in 1:50){
    b0 <- beta0_grid[i]
    b1 <- beta1_grid[j]

    rss_values[i,j] <- sum((y - b0 - b1*x)^2)
  }
}

min(rss_values)
## [1] 34.06858
```

Conclusion

The RSS is minimized at the least squares estimates:

$$\widehat{\beta}_0 = 2.0514, \hat{\beta} = 2.94119$$

Thus, least squares estimators give the best fitting line by minimizing RSS.

3. Problem to demonstrate that least square estimators are unbiased

We take:

- $\beta_0 = 2$
- $\beta = 3$
- Repeat simulation $R = 1000$ times

Step 1–2: Simulation

```
n <- 50
R <- 1000

beta0 <- 2
beta1 <- 3

beta0_est <- numeric(R)
beta1_est <- numeric(R)

for(r in 1:R){

  x <- runif(n, 0, 1)
  eps <- rnorm(n, 0, 1)

  y <- beta0 + beta1*x + eps

  model <- lm(y ~ x)

  beta0_est[r] <- coef(model)[1]
  beta1_est[r] <- coef(model)[2]
}

mean(beta0_est)
## [1] 2.013673

mean(beta1_est)
## [1] 2.981008
```

Comparison with True Values

```
cat("True beta0 =", beta0, "\n")
## True beta0 = 2

cat("Average estimated beta0 =", mean(beta0_est), "\n\n")
## Average estimated beta0 = 2.013673

cat("True beta1 =", beta1, "\n")
## True beta1 = 3

cat("Average estimated beta1 =", mean(beta1_est), "\n")
## Average estimated beta1 = 2.981008
```

Comment

The average estimated values are very close to the true parameters:

$$\beta_0 = 2, \quad \beta = 3$$

Thus, least squares estimators are unbiased.

4. Comparing Several Simple Linear Regressions

We use the Boston dataset

Response:

- medv

Predictors:

- crim
- nox
- black
- lstat

(a) Run Four Separate Linear Regressions

```
library(MASS)

## Warning: package 'MASS' was built under R version 4.5.2

library(stargazer)

## Warning: package 'stargazer' was built under R version 4.5.2

##
## Please cite as:
## Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary
## Statistics Tables.
## R package version 5.2.3. https://CRAN.R-project.org/package=stargazer

data("Boston")

m1 <- lm(medv ~ crim, data=Boston)
m2 <- lm(medv ~ nox, data=Boston)
m3 <- lm(medv ~ black, data=Boston)
m4 <- lm(medv ~ lstat, data=Boston)

stargazer(m1, m2, m3, m4,
           type="text",
           title="Comparison of Four Simple Linear Regression Models",
```



```

dep.var.labels="Median Home Value (medv)",
column.labels=c("crim","nox","black","lstat"),
digits=3)

##
## Comparison of Four Simple Linear Regression Models
## =====
##                               Dependent variable:
##                               -----
##                               Median Home Value (medv)
##                               crim      nox      black      lstat
##                               (1)      (2)      (3)      (4)
##                               -----
## crim                          -0.415***
##                               (0.044)
##
## nox                           -33.916***
##                               (3.196)
##
## black                         0.034***
##                               (0.004)
##
## lstat                         -0.950***
##                               (0.039)
##
## Constant                      24.033*** 41.346*** 10.551*** 34.554***
##                               (0.409) (1.811) (1.557) (0.563)
##
## -----
## Observations                   506      506      506      506
## R2                             0.151      0.183      0.111      0.544
## Adjusted R2                   0.149      0.181      0.109      0.543
## Residual Std. Error (df = 504) 8.484      8.323      8.679      6.216
## F Statistic (df = 1; 504)      89.486*** 112.591*** 63.054*** 601.618***
## =====
## Note:                          *p<0.1; **p<0.05; ***p<0.01

```

(b) Best Fit Model

```

summary(m1)$r.squared
## [1] 0.1507805
summary(m2)$r.squared
## [1] 0.182603
summary(m3)$r.squared
## [1] 0.1111961
summary(m4)$r.squared

```

```
## [1] 0.5441463
```

Answer

The model with predictor **lstat** has the highest R-squared value. Thus, it provides the best fit.

(c) Coefficient Comparison and Usefulness

```
coef(m1)

## (Intercept)      crim
## 24.0331062    -0.4151903

coef(m2)

## (Intercept)      nox
## 41.34587      -33.91606

coef(m3)

## (Intercept)      black
## 10.55103414    0.03359306

coef(m4)

## (Intercept)      lstat
## 34.5538409    -0.9500494
```

Comment

Overall, the four simple linear regression models show that each predictor has a different level of influence on the median housing value (medv). Among the variables considered, lstat (percentage of lower status population) provides the strongest explanatory power, as it produces the highest R-squared value. This indicates that lstat is the most effective single predictor of housing prices in the Boston dataset.

The results also suggest that factors such as higher crime rates (crim) and greater air pollution (nox) are associated with a decline in home values. On the other hand, the predictor black shows a comparatively weaker relationship with the response variable.

Thus, based on both model fit and coefficient strength, lstat emerges as the most useful predictor, and it plays a major role in explaining variations in median home prices.

Thus, **lstat is the most useful predictor** among the four.
